

Практическое задание № 2. Нейронные сети

Работу выполнила:

Охрименко Ева Дмитриевна 409290

Прогнозирование цены на жилье с помощью нейросетевой регрессионной модели

Необходимо по имеющимся данным о ценах на жильё предсказать окончательную цену каждого дома с учетом характеристик домов с использованием нейронной сети. Описание набора данных содержит 80 классов (набор переменных) классификации оценки типа жилья, и находится в файле `data_description.txt`.

В работе требуется дополнить раздел «Моделирование» в подразделе «Построение и обучение модели» создать и инициализировать последовательную модель нейронной сети с помощью фреймворков тренировки нейронных сетей как: Torch или Tensorflow. Скомпилировать нейронную сеть выбрав функцию потерь и оптимизатор соответственно. Оценить точность полученных результатов. Вывести предсказанные данные о продаже.

Импорт библиотек

Импортируем необходимые библиотеки:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from google.colab import drive
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Input
from sklearn.metrics import mean_absolute_error, mean_squared_error
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

Считываем набор данных

Загрузим набор данных и присвоим следующими переменные:

- `train_data`: данные, используемые для обучения модели
- `test_data`: данные, используемые для проверки модели

```
train_data = pd.read_csv('/content/drive/MyDrive/notebook_files/train.csv')
test_data = pd.read_csv('/content/drive/MyDrive/notebook_files/test.csv')
```

Подготовка данных

Отобразим обучающие и проверочные данные:

train_data.head()

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	M
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	

5 rows × 81 columns

```
test_data.head()
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	ScreenPorch	PoolArea	...
0	1461	20	RH	80.0	11622	Pave	NaN	Reg	Lvl	AllPub	...	120	0	
1	1462	20	RL	81.0	14267	Pave	NaN	IR1	Lvl	AllPub	...	0	0	
2	1463	60	RL	74.0	13830	Pave	NaN	IR1	Lvl	AllPub	...	0	0	
3	1464	60	RL	78.0	9978	Pave	NaN	IR1	Lvl	AllPub	...	0	0	
4	1465	120	RL	43.0	5005	Pave	NaN	IR1	HLS	AllPub	...	144	0	

5 rows × 80 columns

Как можно видеть, `train_data` имеет на один столбец больше, чем `test_data`, это столбец `SalePrice`, для обучения модели перед применением ее для предсказания меток в `test_data`.

✓ Проверяем нет ли тестовые данные пустых значений значений (Nan)

Построим функцию `def missing_value_checker` для проверки и подсчёта пропущенных значений в `test_data`. А также выведем тип данных этих значений.

```
def missing_value_checker(data):
    list = []
    for feature, content in data.items():
        if data[feature].isnull().values.any():

            sum = data[feature].isna().sum()

            type = data[feature].dtype

            print (f'{feature}: {sum}, type: {type}')

        list.append(feature)
    print(list)

    print(len(list))

missing_value_checker(test_data)
```

MSZoning: 4, type: object
LotFrontage: 227, type: float64
Alley: 1352, type: object
Utilities: 2, type: object
Exterior1st: 1, type: object
Exterior2nd: 1, type: object
MasVnrType: 894, type: object
MasVnrArea: 15, type: float64
BsmtQual: 44, type: object
BsmtCond: 45, type: object
BsmtExposure: 44, type: object
BsmtFinType1: 42, type: object
BsmtFinSF1: 1, type: float64
BsmtFinType2: 42, type: object
BsmtFinSF2: 1, type: float64
BsmtUnfSF: 1, type: float64
TotalBsmtSF: 1, type: float64
BsmtFullBath: 2, type: float64
BsmtHalfBath: 2, type: float64
KitchenQual: 1, type: object
Functional: 2, type: object
FireplaceQu: 730, type: object
GarageType: 76, type: object
GarageYrBlt: 78, type: float64
GarageFinish: 78, type: object
GarageCars: 1, type: float64
GarageArea: 1, type: float64
GarageQual: 78, type: object
GarageCond: 78, type: object
PoolQC: 1456, type: object
Fence: 1169, type: object
MiscFeature: 1408, type: object
SaleType: 1, type: object
['MSZoning', 'LotFrontage', 'Alley', 'Utilities', 'Exterior1st', 'Exterior2nd', 'MasVnrType', 'MasVnrArea', 'BsmtQual', 'BsmtC
33

Проверяем какие признаки в таблице можно оставить, а какие удалить. Если пропущенных значений слишком много, то удалим признак. Если их небольшое количество, то заполним `mean` или `median` для чисел, новая категория `missing` для строковых объектов.

В соответствии с этим:

- удалим ['Alley', 'FireplaceQu', 'PoolQC', 'Fence', 'MiscFeature'];
- заполним числовое отсутствующее значение значением `mean`;

- заполним строковое отсутствующее значение значением `missing`.

```
test_edited = test_data.drop(['Alley','FireplaceQu','PoolQC', 'Fence', 'MiscFeature'], axis=1)
train_edited = train_data.drop(['Alley','FireplaceQu','PoolQC', 'Fence', 'MiscFeature'], axis=1)

def nan_filler(data):
    for label, content in data.items():
        if pd.api.types.is_numeric_dtype(content):
            data[label] = content.fillna(content.median())
        else:
            data[label] = content.astype("category").cat.as_ordered()
            data[label] = pd.Categorical(content).codes+1

nan_filler(test_edited)
nan_filler(train_edited)
```

✓ Перепроверим наши данные:

```
missing_value_checker(test_edited)
```

```
[]
0
```

```
missing_value_checker(train_edited)
```

```
[]
0
```

```
train_edited.shape, test_edited.shape
```

```
((1460, 76), (1459, 75))
```

```
test_edited.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1459 entries, 0 to 1458
Data columns (total 75 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                    1459 non-null   int64
1   MSSubClass            1459 non-null   int64
2   MSZoning              1459 non-null   int8
3   LotFrontage          1459 non-null   float64
4   LotArea              1459 non-null   int64
5   Street               1459 non-null   int8
6   LotShape             1459 non-null   int8
7   LandContour          1459 non-null   int8
8   Utilities            1459 non-null   int8
9   LotConfig            1459 non-null   int8
10  LandSlope            1459 non-null   int8
11  Neighborhood         1459 non-null   int8
12  Condition1           1459 non-null   int8
13  Condition2           1459 non-null   int8
14  BldgType             1459 non-null   int8
15  HouseStyle           1459 non-null   int8
16  OverallQual          1459 non-null   int64
17  OverallCond          1459 non-null   int64
18  YearBuilt            1459 non-null   int64
19  YearRemodAdd         1459 non-null   int64
20  RoofStyle            1459 non-null   int8
21  RoofMatl            1459 non-null   int8
22  Exterior1st         1459 non-null   int8
23  Exterior2nd         1459 non-null   int8
24  MasVnrType          1459 non-null   int8
25  MasVnrArea          1459 non-null   float64
26  ExterQual            1459 non-null   int8
27  ExterCond           1459 non-null   int8
28  Foundation          1459 non-null   int8
29  BsmtQual            1459 non-null   int8
30  BsmtCond            1459 non-null   int8
31  BsmtExposure        1459 non-null   int8
32  BsmtFinType1         1459 non-null   int8
33  BsmtFinSF1          1459 non-null   float64
34  BsmtFinType2         1459 non-null   int8
35  BsmtFinSF2          1459 non-null   float64
36  BsmtUnfSF           1459 non-null   float64
37  TotalBsmtSF         1459 non-null   float64
38  Heating             1459 non-null   int8
39  HeatingQC           1459 non-null   int8
40  CentralAir          1459 non-null   int8
41  Electrical          1459 non-null   int8
42  1stFlrSF            1459 non-null   int64
43  2ndFlrSF            1459 non-null   int64
44  LowQualFinSF        1459 non-null   int64
45  GrLivArea           1459 non-null   int64
46  BsmtFullBath         1459 non-null   float64
47  BsmtHalfBath         1459 non-null   float64
48  FullBath            1459 non-null   int64
49  HalfBath            1459 non-null   int64
```

```
50 BedroomAbvGr 1459 non-null int64
51 KitchenAbvGr 1459 non-null int64
```

```
train_edited.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 76 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                    1460 non-null   int64
1   MSSubClass            1460 non-null   int64
2   MSZoning              1460 non-null   int8
3   LotFrontage          1460 non-null   float64
4   LotArea              1460 non-null   int64
5   Street               1460 non-null   int8
6   LotShape             1460 non-null   int8
7   LandContour          1460 non-null   int8
8   Utilities            1460 non-null   int8
9   LotConfig            1460 non-null   int8
10  LandSlope            1460 non-null   int8
11  Neighborhood         1460 non-null   int8
12  Condition1           1460 non-null   int8
13  Condition2           1460 non-null   int8
14  BldgType             1460 non-null   int8
15  HouseStyle           1460 non-null   int8
16  OverallQual          1460 non-null   int64
17  OverallCond          1460 non-null   int64
18  YearBuilt            1460 non-null   int64
19  YearRemodAdd         1460 non-null   int64
20  RoofStyle            1460 non-null   int8
21  RoofMatl            1460 non-null   int8
22  Exterior1st         1460 non-null   int8
23  Exterior2nd         1460 non-null   int8
24  MasVnrType          1460 non-null   int8
25  MasVnrArea          1460 non-null   float64
26  ExterQual            1460 non-null   int8
27  ExterCond            1460 non-null   int8
28  Foundation           1460 non-null   int8
29  BsmtQual            1460 non-null   int8
30  BsmtCond            1460 non-null   int8
31  BsmtExposure        1460 non-null   int8
32  BsmtFinType1        1460 non-null   int8
33  BsmtFinSF1          1460 non-null   int64
34  BsmtFinType2        1460 non-null   int8
35  BsmtFinSF2          1460 non-null   int64
36  BsmtUnfSF           1460 non-null   int64
37  TotalBsmtSF         1460 non-null   int64
38  Heating             1460 non-null   int8
39  HeatingQC           1460 non-null   int8
40  CentralAir          1460 non-null   int8
41  Electrical           1460 non-null   int8
42  1stFlrSF            1460 non-null   int64
43  2ndFlrSF            1460 non-null   int64
44  LowQualFinSF        1460 non-null   int64
45  GrLivArea           1460 non-null   int64
46  BsmtFullBath        1460 non-null   int64
47  BsmtHalfBath        1460 non-null   int64
48  FullBath            1460 non-null   int64
49  HalfBath            1460 non-null   int64
50  BedroomAbvGr       1460 non-null   int64
51  KitchenAbvGr       1460 non-null   int64
52  KitchenQual         1460 non-null   int8
```

Разделим данные

Поскольку мы не знаем метку (Цена) тестовых данных, для оценки модели, чтобы получить лучшую модель перед прогнозированием тестовых данных, разделим данные в файле train.scv на обучающие и проверочные данные, соотношение составляет 20%.

```
X = train_edited.drop('SalePrice', axis=1)
y = train_edited['SalePrice']

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size = 0.2)
```

```
X_train.shape, test_edited.shape
```

```
((1168, 75), (1459, 75))
```

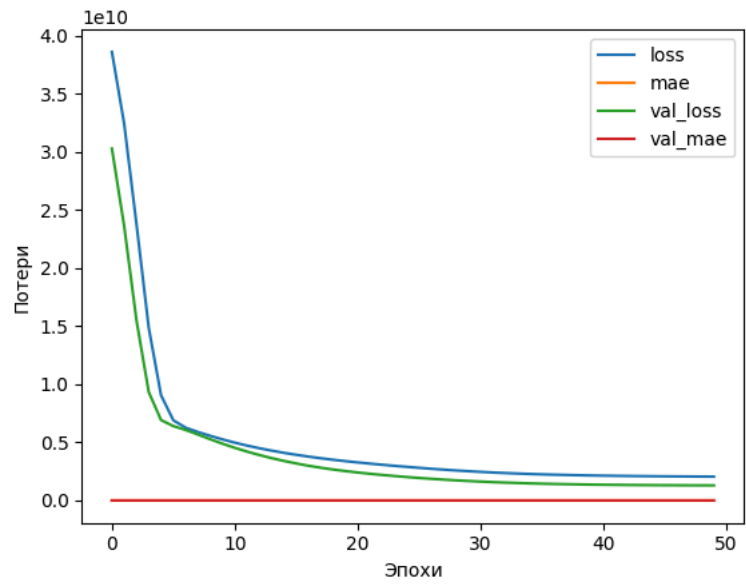
Моделирование

Построение и обучение модели

Создайте последовательную модель нейронной сети с помощью фреймворков тренировки нейронных сетей как: Torch или Tensorflow.


```
pd.DataFrame(history.history).plot()
plt.ylabel('Потери')
plt.xlabel('Эпохи')
print(history.history)
```

{'loss': [38621949952.0, 32434538496.0, 23806760960.0, 14864507904.0, 9061199872.0, 6902675968.0, 6265568768.0, 5895995392.0, ...]}



```
scores = model.evaluate(X_val, y_val, verbose=0) # Оценка модели на валидационных данных
print(f"Потеря (val.): {scores[0]:.2f}")
print(f"MAE (val.): {scores[1]:.2f}")
```

Потеря (val.): 1297196544.00
MAE (val.): 24864.69

▼ Прогнозирование

```
preds = model.predict(test_edited) # Прогнозирование на тестовых данных
preds
```

46/46 ————— **0s** 2ms/step
array([[143902.5],
 [179029.55],
 [194538.03],
 ...,
 [195779.67],
 [99117.805],
 [222116.88]], dtype=float32)

```
#пример
output = pd.DataFrame(
{
    'Id':test_data['Id'],
    'SalePrice': np.squeeze(preds)
})
output
```

	Id	SalePrice
0	1461	143902.500000
1	1462	179029.546875
2	1463	194538.031250
3	1464	195184.562500
4	1465	164102.312500
...	...	...
1454	2915	77210.304688
1455	2916	95226.039062
1456	2917	195779.671875
1457	2918	99117.804688
1458	2919	222116.875000

1459 rows × 2 columns

Задание

В задние представлено логика выполнения с использование tensorflow/keras. Выполнять можно как с использованием tensorflow/keras, так и pytorch.

При выполнении:

- ☐ Выведите отчет нейросетевой регрессионной модели, для прогнозирование цены на жилье.

```
print("Информация о модели:")
print("Входной слой: 256 нейронов")
print("Выходной слой: 1 нейрон")
print(f"\nРезультаты на валидационных данных:")
print(f"MSE: {scores[0]:.2f}")
print(f"MAE: {scores[1]:.2f}")

# Анализ предсказаний
y_pred = model.predict(X_val)
residuals = y_val - y_pred.flatten()
print(f"\nСтатистика остатков:")
print(f"Средний остаток: {np.mean(residuals):.2f}")
print(f"Стандартное отклонение остатков: {np.std(residuals):.2f}")
```

Информация о модели:
Входной слой: 256 нейронов
Выходной слой: 1 нейрон

Результаты на валидационных данных:
MSE: 1297196544.00
MAE: 24864.69
10/10 ————— **0s** 4ms/step

Статистика остатков:
Средний остаток: -6064.05
Стандартное отклонение остатков: 35502.45

Дополнительные функции

```
def plots(name, df_results):

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))

    # 1. MAE
    axes[0, 0].plot(df_results[name], df_results['mae'], 'b-')
    axes[0, 0].set_xscale('log')
    axes[0, 0].set_xlabel(name)
    axes[0, 0].set_ylabel('MAE')
    axes[0, 0].set_title('MAE')
    axes[0, 0].grid(True)

    # 2. MSE
    axes[0, 1].plot(df_results[name], df_results['mse'], 'g-')
    axes[0, 1].set_xscale('log')
    axes[0, 1].set_xlabel(name)
    axes[0, 1].set_ylabel('MSE')
    axes[0, 1].set_title('MSE')
    axes[0, 1].grid(True)

    # 3. Train Loss
    axes[0, 2].plot(df_results[name], df_results['loss_train'], 'r-')
    axes[0, 2].set_xscale('log')
    axes[0, 2].set_xlabel(name)
    axes[0, 2].set_ylabel('Train Loss')
    axes[0, 2].set_title('Train Loss')
    axes[0, 2].grid(True)

    # 4. Validation Loss
    axes[1, 0].plot(df_results[name], df_results['loss_val'], 'm-')
    axes[1, 0].set_xscale('log')
    axes[1, 0].set_xlabel(name)
    axes[1, 0].set_ylabel('Val Loss')
    axes[1, 0].set_title('Validation Loss')
    axes[1, 0].grid(True)

    # 5. Train MAE
    axes[1, 1].plot(df_results[name], df_results['mae_train'], 'c-')
    axes[1, 1].set_xscale('log')
    axes[1, 1].set_xlabel(name)
    axes[1, 1].set_ylabel('Train MAE')
    axes[1, 1].set_title('Train MAE')
    axes[1, 1].grid(True)

    # 6. Validation MAE
    axes[1, 2].plot(df_results[name], df_results['mae_val'], 'y-')
    axes[1, 2].set_xscale('log')
    axes[1, 2].set_xlabel(name)
    axes[1, 2].set_ylabel('Val MAE')
```

```
axes[1, 2].set_title('Validation MAE')
axes[1, 2].grid(True)
```

```
plt.tight_layout()
plt.show()
```

```
def get_results(neurons_vhod=[64], neurons_skrit=[0], batch_size=[32], epoch_count=[50], optimizers=['adam'], losses=['mse'])

    results = []

    for neurons1 in neurons_vhod:
        for neurons2 in neurons_skrit:
            for batch in batch_size:
                for epoch in epoch_count:
                    for optimizer_name in optimizers:
                        for loss_name in losses:

                            model = Sequential([
                                Input(shape=(X_train.shape[1],)),
                                Dense(neurons1, activation='linear'),
                            ])

                            if neurons2 > 0:
                                model.add(Dense(neurons2, activation='linear'))

                            model.add(Dense(1))

                            if optimizer_name == 'adam':
                                optimizer = keras.optimizers.Adam(learning_rate=0.01)
                            elif optimizer_name == 'rmsprop':
                                optimizer = keras.optimizers.RMSprop(learning_rate=0.001)
                            elif optimizer_name == 'adagrad':
                                optimizer = keras.optimizers.Adagrad(learning_rate=0.01)
                            elif optimizer_name == 'adamax':
                                optimizer = keras.optimizers.Adamax(learning_rate=0.002)
                            elif optimizer_name == 'nadam':
                                optimizer = keras.optimizers.Nadam(learning_rate=0.002)
                            else:
                                optimizer = keras.optimizers.Adam(learning_rate=0.001)

                            model.compile(
                                optimizer=optimizer,
                                loss=loss_name,
                                metrics=['mae']
                            )

                            history = model.fit(
                                X_train, y_train,
                                epochs=epoch,
                                batch_size=batch,
                                validation_data=(X_val, y_val),
                                verbose=0
                            )

                            y_pred = model.predict(X_val).flatten()

                            mae = mean_absolute_error(y_val, y_pred)
                            mse = mean_squared_error(y_val, y_pred)

                            results.append({
                                'нейроны_входа': neurons1,
                                'нейроны_скрытые': neurons2,
                                'batch_size': batch,
                                'эпохи': epoch,
                                'оптимизатор': optimizer_name,
                                'функция_потерь': loss_name,
                                'mae': mae,
                                'mse': mse,
                                'loss_train': history.history['loss'][-1],
                                'loss_val': history.history['val_loss'][-1],
                                'mae_train': history.history['mae'][-1],
                                'mae_val': history.history['val_mae'][-1]
                            })

    return pd.DataFrame(results)
```

- ☐ Попробуйте использовать разное количество нейронов на входном слое. Опишите достигнутый результат.

```
df_results_vhod = get_results(neurons_vhod=[2, 4, 8, 16, 32, 64, 128, 256, 512])
```

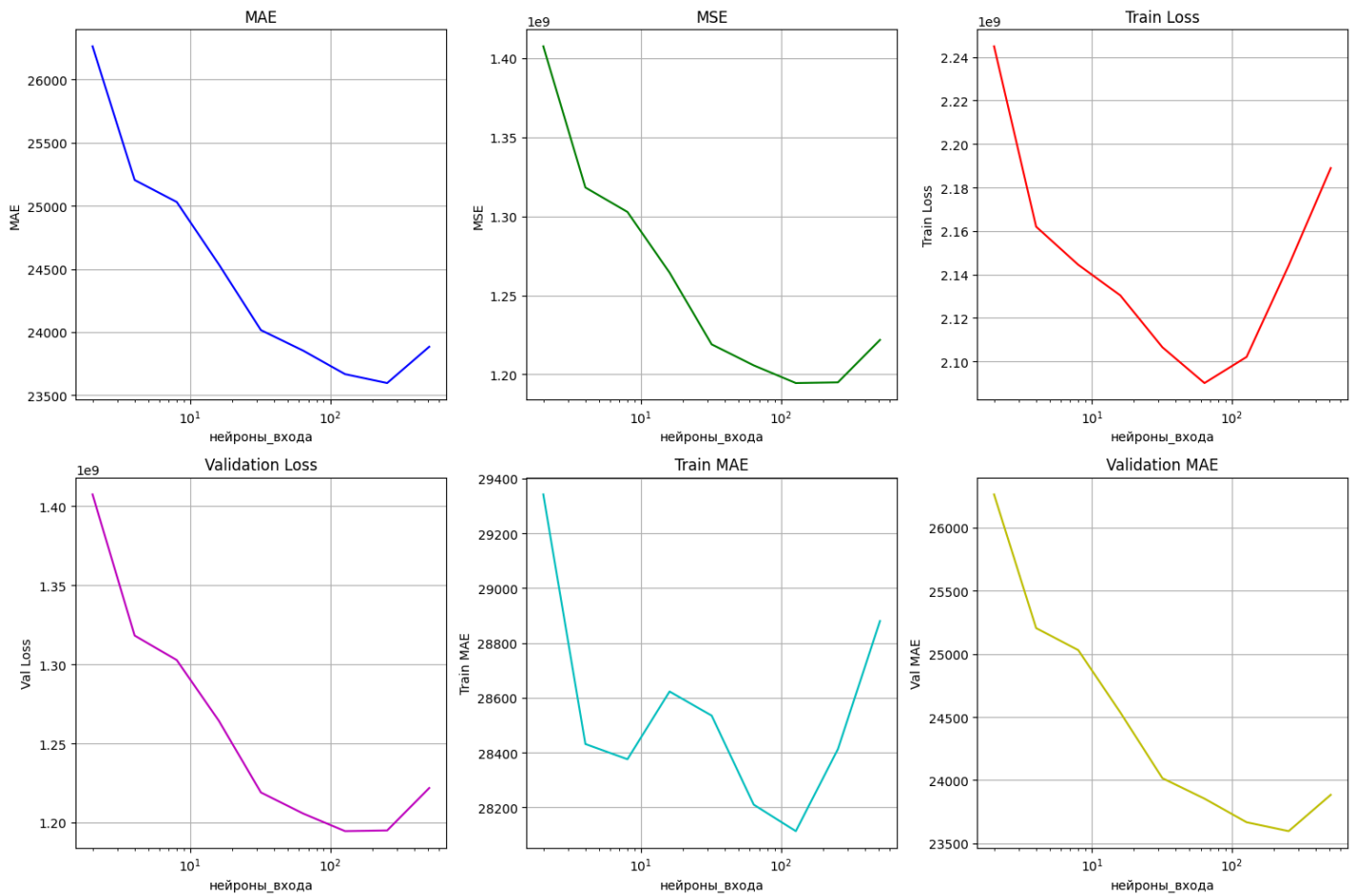
```
10/10 ————— 0s 6ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step
```


10/10 — 0s 7ms/step
10/10 — 0s 5ms/step
10/10 — 0s 5ms/step
10/10 — 0s 5ms/step
10/10 — 0s 5ms/step

df_results_vhod

	нейроны_входа	нейроны_скрытые	batch_size	эпохи	оптимизатор	функция_потерь	mae	mse	loss_train	loss_val
0	2	0	32	50	adam	mse	26264.271484	1.407450e+09	2.244884e+09	1.407450e+09
1	4	0	32	50	adam	mse	25204.671875	1.318182e+09	2.161988e+09	1.318182e+09
2	8	0	32	50	adam	mse	25030.500000	1.302712e+09	2.144488e+09	1.302712e+09
3	16	0	32	50	adam	mse	24536.566406	1.264362e+09	2.130393e+09	1.264362e+09
4	32	0	32	50	adam	mse	24015.322266	1.219007e+09	2.106566e+09	1.219007e+09
5	64	0	32	50	adam	mse	23853.326172	1.205725e+09	2.090106e+09	1.205725e+09
6	128	0	32	50	adam	mse	23666.283203	1.194542e+09	2.102107e+09	1.194542e+09
7	256	0	32	50	adam	mse	23595.646484	1.194975e+09	2.144122e+09	1.194975e+09
8	512	0	32	50	adam	mse	23883.486328	1.221820e+09	2.188919e+09	1.221820e+09

plots('нейроны_входа', df_results_vhod)



- ☐ Добавьте в нейронную сеть скрытый слой с разным количеством нейронов.

df_results_skrit = get_results(neurons_skrit=[2, 4, 8, 16, 32, 64, 128, 256, 512])

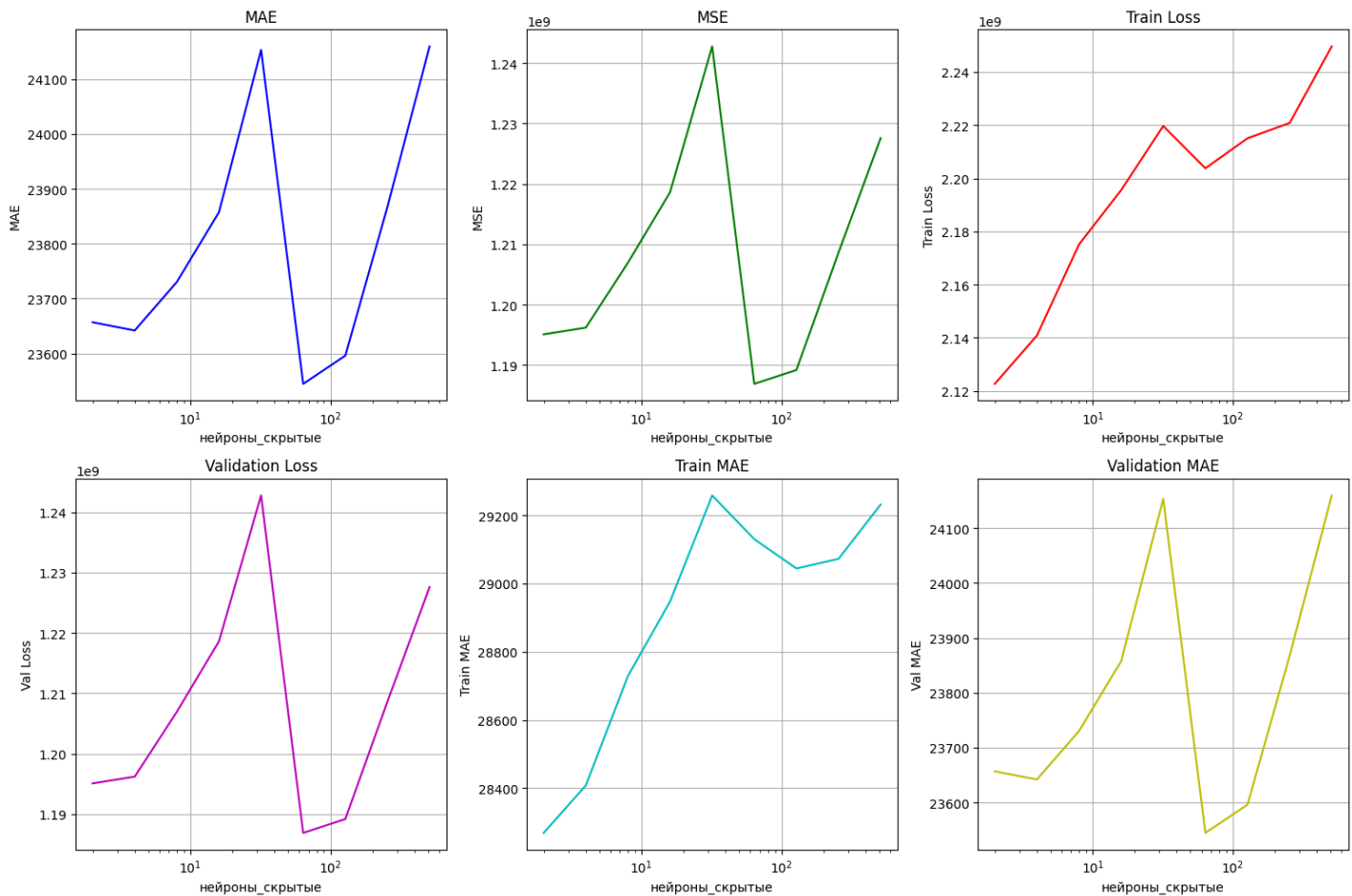
10/10 — 0s 6ms/step
10/10 — 0s 6ms/step

10/10 0s 7ms/step
10/10 0s 6ms/step
10/10 1s 6ms/step
10/10 0s 6ms/step
10/10 0s 6ms/step
10/10 0s 6ms/step
10/10 0s 6ms/step
10/10 0s 6ms/step

df_results_skrit

	нейроны_входа	нейроны_скрытые	batch_size	эпохи	оптимизатор	функция_потерь	mae	mse	loss_train	loss_val
0	64	2	32	50	adam	mse	23656.886719	1.195082e+09	2.122651e+09	1.195082e+09
1	64	4	32	50	adam	mse	23641.968750	1.196196e+09	2.140822e+09	1.196196e+09
2	64	8	32	50	adam	mse	23730.380859	1.206942e+09	2.175113e+09	1.206942e+09
3	64	16	32	50	adam	mse	23857.509766	1.218608e+09	2.195544e+09	1.218608e+09
4	64	32	32	50	adam	mse	24153.363281	1.242769e+09	2.219693e+09	1.242769e+09
5	64	64	32	50	adam	mse	23544.605469	1.186879e+09	2.203762e+09	1.186879e+09
6	64	128	32	50	adam	mse	23596.093750	1.189176e+09	2.215068e+09	1.189176e+09
7	64	256	32	50	adam	mse	23867.253906	1.208624e+09	2.220804e+09	1.208624e+09
8	64	512	32	50	adam	mse	24159.708984	1.227590e+09	2.249633e+09	1.227590e+09

plots('нейроны_скрытые', df_results_skrit)



- ☐ Используйте разные размеры мини-выборки (batch_size). Опишите достигнутый результат.

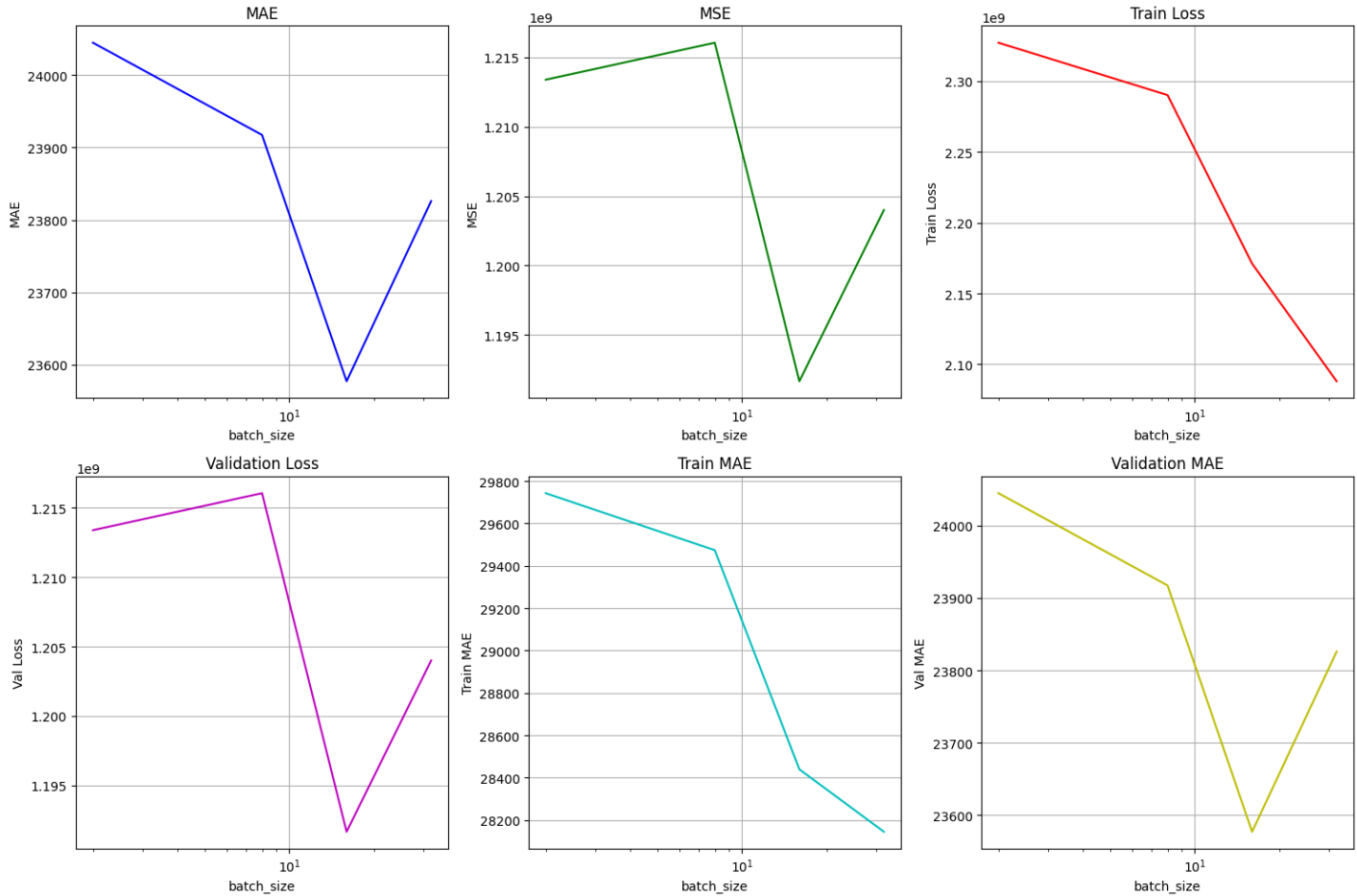
df_results_batch = get_results(batch_size=[2, 8, 16, 32])

```
10/10 ————— 0s 5ms/step
10/10 ————— 0s 7ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step
```

df_results_batch

	нейроны_входа	нейроны_скрытые	batch_size	эпохи	оптимизатор	функция_потерь	mae	mse	loss_train	loss_val
0	64	0	2	50	adam	mse	24044.687500	1.213379e+09	2.327132e+09	1.213379e+09
1	64	0	8	50	adam	mse	23917.326172	1.216042e+09	2.290145e+09	1.216042e+09
2	64	0	16	50	adam	mse	23577.195312	1.191671e+09	2.171110e+09	1.191671e+09
3	64	0	32	50	adam	mse	23826.013672	1.203999e+09	2.088026e+09	1.203999e+09

plots('batch_size', df_results_batch)



- ☐ Используйте разное количество эпох. Опишите достигнутый результат.

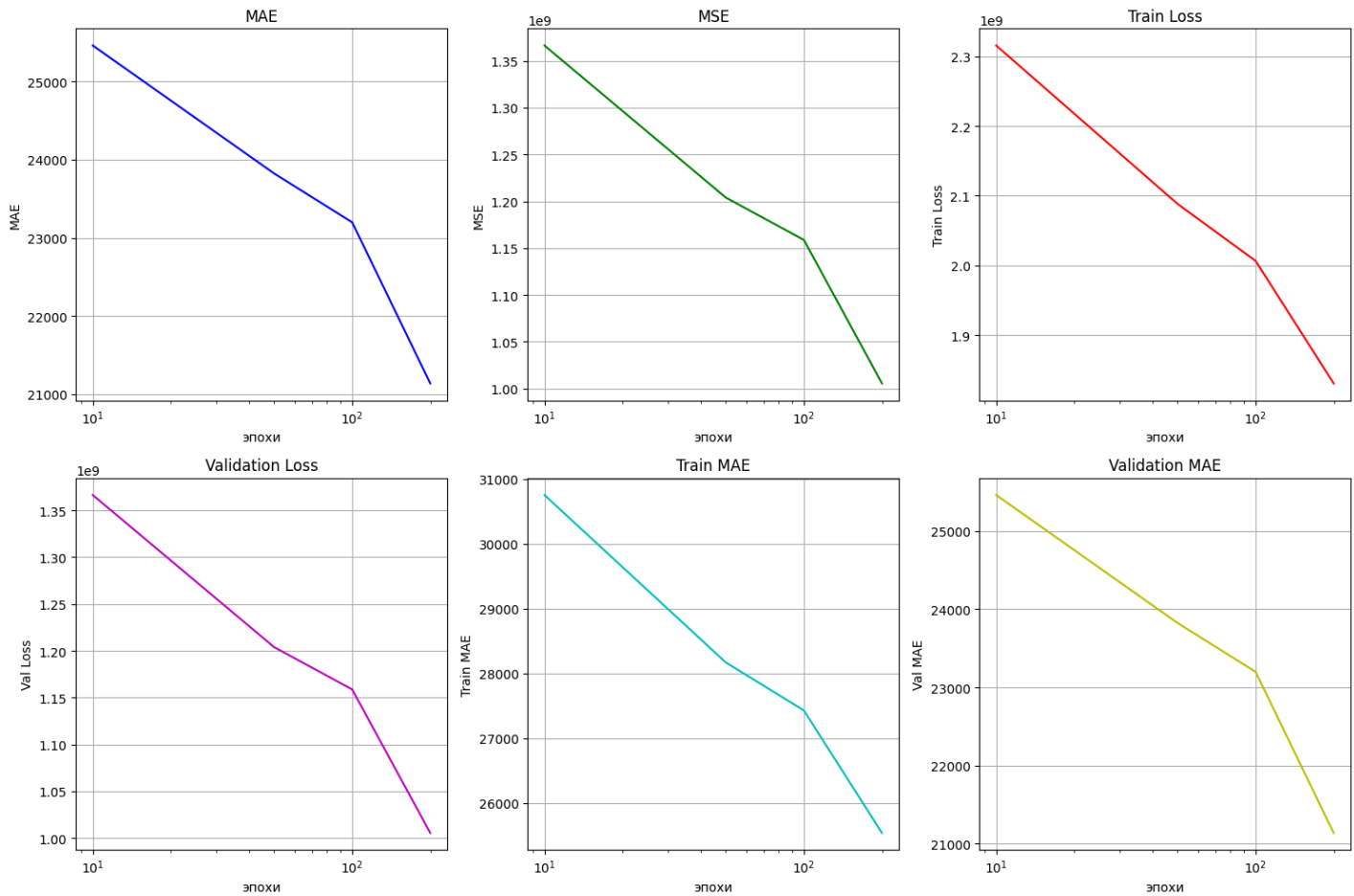
df_results_epoch = get_results(epoch_count=[10, 50, 100, 200])

```
10/10 ————— 0s 6ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step
```

df_results_epoch

	нейроны_входа	нейроны_скрытые	batch_size	эпохи	оптимизатор	функция_потерь	mae	mse	loss_train	loss_val
0	64	0	32	10	adam	mse	25459.009766	1.366445e+09	2.315610e+09	1.366445e+09
1	64	0	32	50	adam	mse	23823.498047	1.203972e+09	2.088391e+09	1.203972e+09
2	64	0	32	100	adam	mse	23198.582031	1.158723e+09	2.006620e+09	1.158723e+09
3	64	0	32	200	adam	mse	21138.255859	1.005366e+09	1.830755e+09	1.005366e+09

plots('эпохи', df_results_epoch)



- ☐ Попробуйте использовать разные значения оптимизатора (`optimizers`). Сравните полученные результаты.

Adam

- Запоминает скользящие средние градиентов и их квадратов, автоматически адаптирует шаг для каждого веса.

RMSprop

- Большие градиенты - уменьшает шаг, маленькие - увеличивает.

Adamax

- Использует максимум градиентов вместо среднего квадрата, более устойчив к редким обновлениям. (видимо другая функция ошибки, не просто l2)

Nadam

- Делает "просмотр" в направлении градиента перед обновлением весов

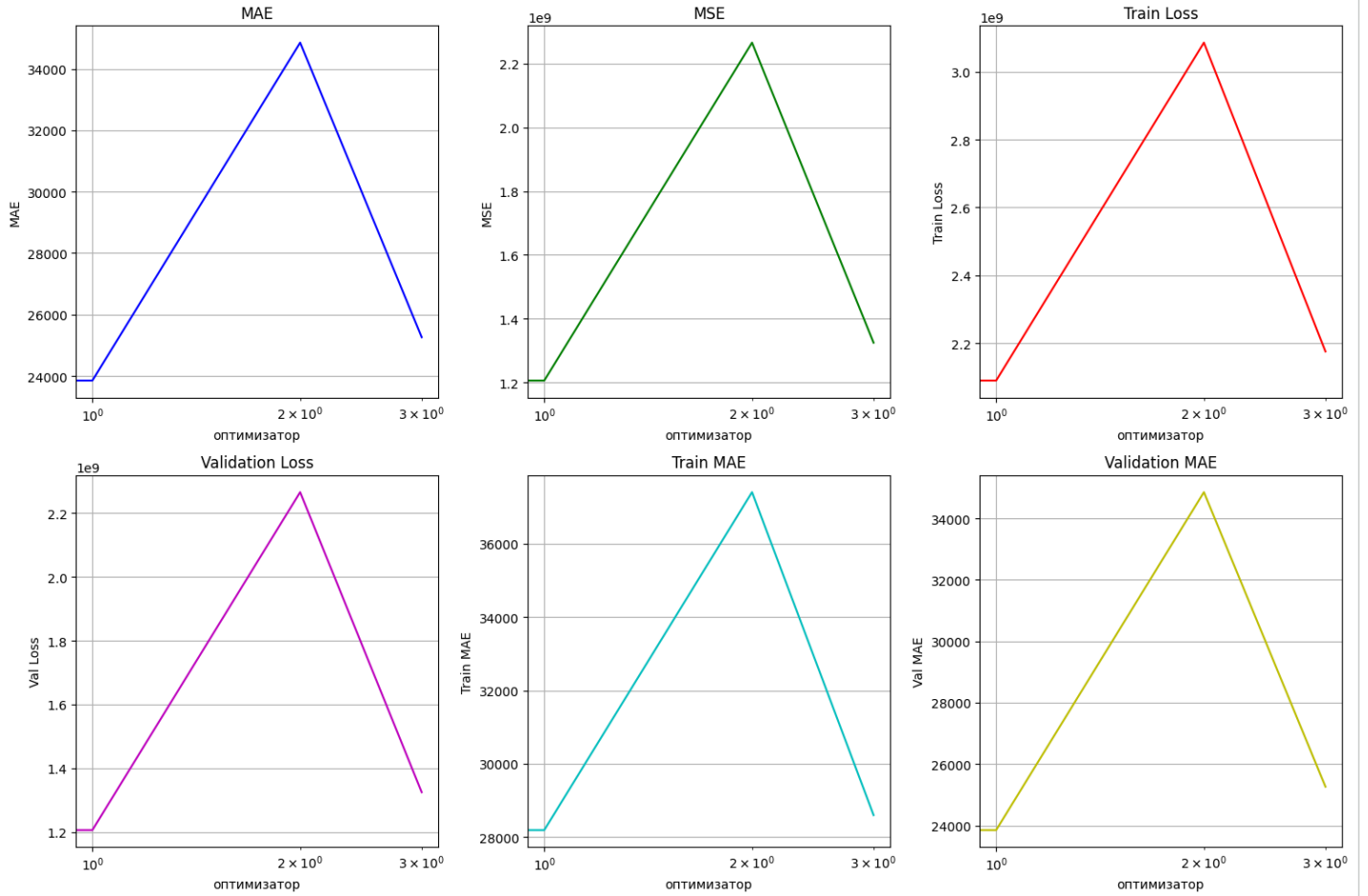
```
df_results_optim = get_results(optimizers=['rmsprop','adam', 'adamax', 'nadam'])
```

10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 1s 7ms/step

df_results_optim

	нейроны_входа	нейроны_скрытые	batch_size	эпохи	оптимизатор	функция_потерь	mae	mse	loss_train	loss_val
0	64	0	32	50	rmsprop	mse	29556.859375	1.686323e+09	2.570745e+09	1.686323e+09
1	64	0	32	50	adam	mse	23849.841797	1.206004e+09	2.089778e+09	1.206004e+09
2	64	0	32	50	adamax	mse	34856.089844	2.265526e+09	3.085977e+09	2.265526e+09
3	64	0	32	50	nadam	mse	25262.500000	1.324728e+09	2.175532e+09	1.324728e+09

plots('оптимизатор', df_results_optim)



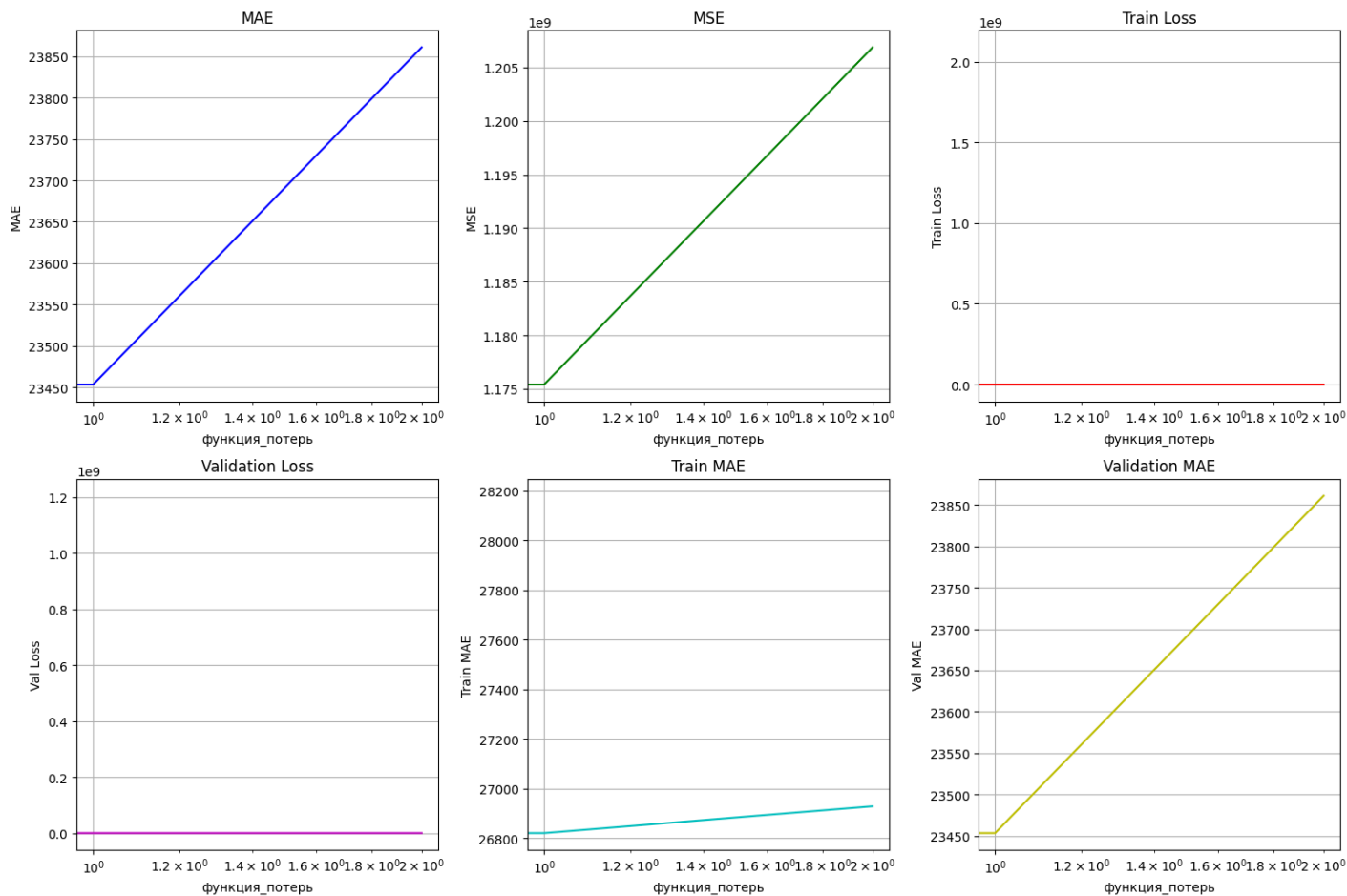
df_results_loss = get_results(losses=['mse', 'mae', 'huber'])

10/10 ————— 0s 6ms/step
10/10 ————— 0s 5ms/step
10/10 ————— 0s 5ms/step

df_results_loss

	нейроны_входа	нейроны_скрытые	batch_size	эпохи	оптимизатор	функция_потерь	mae	mse	loss_train	loss_val
0	64	0	32	50	adam	mse	23841.275391	1.205100e+09	2.089155e+09	1.205100e+09
1	64	0	32	50	adam	mae	23453.187500	1.175408e+09	2.682011e+04	2.345319e+04
2	64	0	32	50	adam	huber	23861.021484	1.206878e+09	2.692766e+04	2.386052e+04

```
plots('функция_потерь', df_results_loss)
```



Вопросы:

Как выше перечисленные параметры влияют на полученный вами результат?

Что такое эпоха (Epoch)? В чем отличие от итерации (Iteration)?

Что такое функция активации? Какие вам известны? Как и зачем используются в нейронной сети?

Что такое MSE (Mean Squared Error) - Средняя квадратичная ошибка? Что такое MAE (Mean Absolute Error)? Для чего используются.